Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп'ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 3

Тема: Програмування алгоритмів розгалуженої та циклічної структури

Мета роботи: Опанувати практичні навички алгоритмізації та проектування програм розгалуженої та циклічної структури мовою Python. Навчитися реалізовувати багатоваріантний вибір за допомогою повних і каскадних умовних конструкцій if-elif-else, застосовувати тернарні умовні вирази для оптимізації потоку керування, розробляти ітераційні алгоритми обчислення нескінченних числових рядів із заданою точністю
за допомогою циклу з передумовою while з використанням рекурентних співвідношень, а також виконувати чисельне табулювання кусочно-неперервних функцій за допомогою параметричного циклу for та генератора діапазонів range().

Студента III курсу КІ-24-1 групи
Ступінь вищої освіти бакалавр
(бакалавр, магістр)
Спеціальність 123 – «Комп'ютерна інженерія»
Освітня програма Комп'ютерна інженерія
Яцентюка Євгенія Олександровича

м. Кременчук 2026 рік

1. Мета роботи та математична постановка задачі

Мета роботи: опанувати проєктування програм розгалуженої та циклічної структури мовою Python; навчитися застосовувати конструкцію if-elif-else та тернарний умовний вираз, виконувати чисельне табулювання кусочно-неперервної функції циклом for і обчислювати суму нескінченного числового ряду із заданою точністю циклом while з використанням рекурентного співвідношення.

1.1. Індивідуальне завдання (варіант 2)

Частина	**Умова**
Функція	$f(x) = \cos(x^2) + 1$ при $x < -1$; $x^3 + 2x$ при $-1 \le x \le 1$; $\sqrt[3]{x - 1}$ при $x > 1$
Табулювання	відрізок $[-3, 3]$, крок $h = 0{,}25$
Ряд	$\sin(x) = \Sigma\, (-1)^k\, x^{2k+1} / (2k + 1)!$, $k = 0, 1, 2, \ldots$
Точність	$\varepsilon = 10^{-7}$
Еталон	math.sin(x)

1.2. Математичний опис

Кусочно-неперервна функція визначається трьома аналітичними виразами залежно від інтервалу, якому належить аргумент:

$$f(x) = \cos(x^2) + 1, \quad x < -1$$

$$f(x) = x^3 + 2x, \quad -1 \le x \le 1$$

$$f(x) = (x - 1)^{(1/3)}, \quad x > 1$$

У третій гілці $x - 1 > 0$, тому кубічний корінь визначений і дійсний. У точках $x = -1$ та $x = 1$ функція має розриви першого роду: ліва границя в точці -1 дорівнює $\cos(1) + 1 \approx 1{,}5403$, а значення $f(-1) = -3$; значення $f(1) = 3$, а права границя в точці 1 дорівнює 0.

Вузли сітки табулювання та кількість кроків розбиття:

$$x_i = a + i \cdot h, \quad i = 0, 1, \ldots, N, \quad N = \lfloor (b - a) / h \rfloor = \lfloor 6 / 0{,}25 \rfloor = 24$$

Отже, таблиця містить N + 1 = 25 вузлів. Частинна сума ряду Тейлора та умова зупинки ітераційного процесу:

$$S_n(x) = \Sigma_{k=0}^{n} a_k(x), \quad |a_n(x)| < \varepsilon = 10^{-7}$$

2. Виведення рекурентного співвідношення

Загальний член ряду Тейлора для sin(x):

$$a_k(x) = (-1)^k \cdot x^{2k+1} / (2k + 1)!$$

Для попереднього члена (k − 1) отримуємо:

$$a_{k-1}(x) = (-1)^{k-1} \cdot x^{2k-1} / (2k - 1)!$$

Знаходимо відношення сусідніх членів:

$$R(k, x) = a_k / a_{k-1} = (-1) \cdot x^2 \cdot (2k - 1)! / (2k + 1)!$$

Оскільки (2k + 1)! = (2k + 1) · 2k · (2k − 1)!, факторіал (2k − 1)! скорочується:

$$R(k, x) = -x^2 / (2k \cdot (2k + 1))$$

Початковий член: $a_0 = x$ (при k = 0 маємо $(-1)^0 \cdot x^1 / 1! = x$). Остаточно:

$$a_0 = x, \quad a_k = a_{k-1} \cdot (-x^2) / (2k \cdot (2k + 1))$$

Перевірка: $a_1 = x \cdot (-x^2) / (2 \cdot 3) = -x^3 / 6$, що збігається з другим членом розкладу $\sin(x) = x - x^3/3! + x^5/5! - \ldots$

Пряме обчислення кожного члена вимагало б O(k) множень для піднесення до степеня та обчислення факторіала, тому сумарна складність становила б $O(n^2)$. Рекурентна формула визначає наступний член за сталу кількість операцій O(1), отже підсумовування n членів має складність O(n). Крім того, рекурентний підхід уникає переповнення при обчисленні великих факторіалів.

Ряд для sin(x) є знакозмінним. Починаючи з номера, за якого $|R(k, x)| < 1$, модулі членів монотонно спадають, тому похибка відсікання не перевищує модуля першого відкинутого члена. Цикл завершується, коли $|a_k| < \varepsilon$, тож

абсолютна похибка суми має бути меншою за ε. Це підтверджується результатами в розділі 5.

3. Алгоритмічний аналіз

3.1. Розгалужений алгоритм обчислення f(x)

Функція evaluate_piecewise_function перевіряє умови каскадом if-elif-else у порядку $x < -1$, $-1 \le x \le 1$, інакше. Гілки взаємовиключні й покривають усю числову вісь, тому для будь-якого x виконується рівно одна з них. Межові точки $x = -1$ та $x = 1$ належать другій гілці, що відповідає умові із завдання (нерівності нестрогі саме в другому рядку).

Назва гілки визначається окремою функцією classify_branch за допомогою вкладеного тернарного оператора: "Гілка 1" if x < −1 else ("Гілка 2" if x <= 1 else "Гілка 3"). На відміну від оператора if-else, тернарний оператор є виразом і повертає значення, тому його можна використати безпосередньо в оголошенні кортежу результатів.

Для демонстрації короткої схеми обчислення реалізовано функцію reciprocal_exceeds_one(x) з виразом x != 0 and 1 / x > 1. Якщо x = 0, перший операнд кон'юнкції хибний, другий не обчислюється, і ділення на нуль не відбувається.

3.2. Табулювання функції

Функція range() працює лише з цілими числами, тому цикл for перебирає цілочисельний лічильник i від 0 до N, а значення аргументу обчислюється як $x_i = a + i \cdot h$. Такий спосіб не накопичує похибку округлення, на відміну від багаторазового додавання кроку x += h. Кількість кроків N обчислюється через math.floor з малою добавкою 10^{-9}, що захищає від втрати останнього вузла внаслідок похибки подання дійсних чисел.

Формальний опис алгоритму табулювання:

1) перевірити умови $h > 0$ та $a \le b$, інакше згенерувати ValueError;
2) обчислити $N = \lfloor (b - a) / h \rfloor$ та створити порожній список результатів;

3) для i від 0 до N: обчислити $x_i = a + i \cdot h$, $y_i = f(x_i)$ та назву гілки;

4) додати кортеж (x_i, y_i, гілка) до списку та повернути список.

3.3. Ітераційне підсумовування ряду

Цикл while виконується, доки $|a_k| \geq \varepsilon$ та лічильник k менший за max_iterations = 1000. Другу умову введено для захисту від зациклення. Інваріант циклу: перед кожною перевіркою умови змінна series_sum дорівнює $\Sigma_{j=0}^{k} a_j$, а змінна a_k містить k-й член ряду.

Формальний опис алгоритму:

1) встановити k = 0, a = x, sum = a;

2) поки $|a| \geq \varepsilon$ та $k <$ max_iterations: збільшити k на 1, помножити a на $-x^2 / (2k \cdot (2k + 1))$, додати a до sum;

3) блок else циклу виводить попередження, якщо зупинку спричинено ліміт ітерацій, а не досягнення точності;

4) порівняти sum з math.sin(x) і повернути (sum, k, еталон, абсолютна похибка).

Для x = 0 нульовий член $a_0 = 0$ вже менший за ε, тому цикл не виконується жодного разу, а сума дорівнює 0, що збігається з sin(0).

Структура	Реалізація	Складність
Розгалуження	if-elif-else, тернарний оператор	O(1) на обчислення f(x)
Табулювання	for i in range(N + 1)	O(N), N = 24
Ряд Тейлора	while з рекурентною формулою	O(n), n — кількість членів

4. Програмний код

4.1. Файл branching_logic.py

```
"""
Лабораторна робота № 3. Модуль 1. Варіант 2.
Розгалужені алгоритми: обчислення кусочно-неперервної функції y = f(x)
за допомогою if-elif-else, класифікація гілки тернарним оператором,
демонстрація короткої схеми обчислення (short-circuit evaluation).
```

```
        | cos(x^2) + 1,   x < -1
f(x) =  | x^3 + 2x,       -1 <= x <= 1
        | cbrt(x - 1),    x > 1

Освітній компонент: Програмування на Python.
"""

import math
import time

# Межі гілок кусочної функції
LEFT_BOUND: float = -1.0
RIGHT_BOUND: float = 1.0


def evaluate_piecewise_function(x: float) -> float:
    """
    Обчислює f(x) за допомогою каскадної конструкції if-elif-else.
    Повертає дійсне значення функції у точці x.
    """
    if x < LEFT_BOUND:
        # Гілка 1: x < -1
        y = math.cos(x**2) + 1.0
    elif LEFT_BOUND <= x <= RIGHT_BOUND:
        # Гілка 2: -1 <= x <= 1
        y = x**3 + 2.0 * x
    else:
        # Гілка 3: x > 1. Тут x - 1 > 0, тому кубічний корінь визначений.
        y = (x - 1.0) ** (1.0 / 3.0)
    return y


def classify_branch(x: float) -> str:
    """Повертає назву гілки функції для точки x за допомогою вкладеного
тернарного оператора."""
    return "Гілка 1" if x < LEFT_BOUND else ("Гілка 2" if x <= RIGHT_BOUND else
"Гілка 3")


def reciprocal_exceeds_one(x: float) -> bool:
    """
    Перевіряє нерівність 1/x > 1 з використанням короткої схеми обчислення.
    Якщо x == 0, перший операнд кон'юнкції хибний, і ділення 1/x не виконується,
    тому ZeroDivisionError не виникає.
    """
    return x != 0.0 and 1.0 / x > 1.0


def main() -> None:
    """Демонстрація розгалужених обчислень на контрольних точках."""
```

```
    print("=" * 72)
    print(f"{'ВАРІАНТ 2. КУСОЧНО-НЕПЕРЕРВНА ФУНКЦІЯ: РОЗГАЛУЖЕННЯ':^72}")
    print("=" * 72)

    # Контрольні точки: усередині гілок та по обидва боки від меж -1 і 1
    control_points = [-3.0, -1.5, -1.0001, -1.0, 0.0, 0.5, 1.0, 1.0001, 2.0, 3.0]

    print(f"| {'x':^10} | {'Гілка':^10} | {'f(x)':^16} |")
    print("-" * 72)
    for x in control_points:
        y = evaluate_piecewise_function(x)
        print(f"| {x:^10.4f} | {classify_branch(x):^10} | {y:^16.8f} |")
    print("-" * 72)

    # Розриви на межах гілок: різниця між односторонньою границею та значенням у
точці
    # x = -1: границя зліва дорівнює cos(1) + 1, значення f(-1) = -3
    left_limit_m1 = math.cos(1.0) + 1.0
    jump_m1 = left_limit_m1 - evaluate_piecewise_function(-1.0)
    # x = 1: значення f(1) = 3, границя справа дорівнює cbrt(0) = 0
    right_limit_p1 = 0.0
    jump_p1 = right_limit_p1 - evaluate_piecewise_function(1.0)
    print(f"Розрив у точці x = -1: границя зліва - значення = {jump_m1:+.6f}")
    print(f"Розрив у точці x = +1: границя справа - значення = {jump_p1:+.6f}")
    print("Функція кусочно-неперервна: у точках x = -1 та x = 1 має розриви
першого роду.")

    # Коротка схема обчислення
    print("\nКоротка схема обчислення (1/x > 1):")
    for x in (0.0, 0.5, 2.0):
        print(f"  x = {x:4.1f}  ->  {reciprocal_exceeds_one(x)}")
    print("=" * 72)


if __name__ == "__main__":
    t_start = time.perf_counter()
    main()
    elapsed = time.perf_counter() - t_start
    print(f"Час виконання модуля розгалужень: {elapsed * 1000:.3f} мс\n")
```

4.2. Файл series_tabulation.py

```
"""
Лабораторна робота № 3. Модуль 2. Варіант 2.
Чисельне табулювання кусочної функції на [-3, 3] з кроком h = 0.25 (цикл for)
та обчислення ряду Тейлора для sin(x) з точністю eps = 1e-7 (цикл while)
за допомогою рекурентного співвідношення.
```

```
sin(x) = sum_{k=0}^{inf} (-1)^k x^(2k+1) / (2k+1)!
a_0 = x,   a_k = a_{k-1} * (-x^2) / (2k * (2k + 1))

Освітній компонент: Програмування на Python.
"""

import math
import time

from branching_logic import classify_branch, evaluate_piecewise_function

# Параметри індивідуального варіанта 2
A_START: float = -3.0
B_END: float = 3.0
H_STEP: float = 0.25
EPSILON: float = 1e-7
MAX_ITERATIONS: int = 1000


def tabulate_function(start_a: float, end_b: float, step_h: float) ->
list[tuple[float, float, str]]:
    """
    Табулює f(x) на відрізку [a, b] з кроком h параметричним циклом for.
    Вузли обчислюються як x_i = a + i * h, i = 0..N, де N = floor((b - a) / h).
    Повертає список кортежів (x_i, y_i, назва_гілки).
    """
    if step_h <= 0.0:
        raise ValueError("Крок табулювання h повинен бути строго додатним.")
    if start_a > end_b:
        raise ValueError("Початкова межа a не може перевищувати кінцеву межу b.")

    # Мала добавка захищає від втрати останнього вузла через похибку округлення
    num_steps = int(math.floor((end_b - start_a) / step_h + 1e-9))

    table = []
    for i in range(num_steps + 1):
        x_i = start_a + i * step_h
        y_i = evaluate_piecewise_function(x_i)
        table.append((x_i, y_i, classify_branch(x_i)))
    return table


def compute_taylor_sin(
    x: float, epsilon: float = EPSILON, max_iterations: int = MAX_ITERATIONS
) -> tuple[float, int, float, float]:
    """
    Обчислює sin(x) сумуванням ряду Тейлора циклом while з рекурентним
співвідношенням.
    Повертає кортеж (сума_ряду, кількість_ітерацій, еталон_math.sin,
абсолютна_похибка).
```

```
    Цикл завершується, коли |a_k| < epsilon або досягнуто max_iterations.
    """
    if epsilon <= 0.0:
        raise ValueError("Точність epsilon повинна бути строго додатною.")

    k = 0
    a_k = x                 # a_0 = x
    series_sum = a_k

    while math.fabs(a_k) >= epsilon and k < max_iterations:
        k += 1
        # Рекурентний коефіцієнт R(k, x) = -x^2 / (2k * (2k + 1))
        a_k *= (-x * x) / (2.0 * k * (2.0 * k + 1.0))
        series_sum += a_k
    else:
        if k >= max_iterations:
            print(f"Попередження: досягнуто ліміт ітерацій ({max_iterations}) для
x = {x:.4f}!")

    reference = math.sin(x)
    return series_sum, k, reference, math.fabs(series_sum - reference)


def print_tabulation_table(data: list[tuple[float, float, str]]) -> None:
    """Виводить таблицю табулювання."""
    sep = "=" * 65
    print(sep)
    print(f"{'ТАБЛИЦЯ ТАБУЛЮВАННЯ ФУНКЦІЇ f(x) НА [-3, 3], h = 0.25':^65}")
    print(sep)
    print(f"| {'№':^4} | {'Вузол x':^14} | {'Значення y = f(x)':^20} |
{'Сегмент':^10} |")
    print("-" * 65)
    for idx, (x_val, y_val, tag) in enumerate(data, start=1):
        print(f"| {idx:^4} | {x_val:^14.4f} | {y_val:^20.8f} | {tag:^10} |")
    print(sep)


def main() -> None:
    """Демонстрація табулювання та підсумовування ряду."""
    print("\n" + "#" * 80)
    print(f"{'ВАРІАНТ 2. ТАБУЛЮВАННЯ ТА РЯД ТЕЙЛОРА ДЛЯ sin(x) (ЛБ №3)':^80}")
    print("#" * 80 + "\n")

    # 1. Табулювання
    table = tabulate_function(A_START, B_END, H_STEP)
    print_tabulation_table(table)
    print(f"Кількість вузлів: {len(table)}  (N = {len(table) - 1} кроків)")

    # 2. Ряд Тейлора
    print("\n" + "=" * 80)
    print(f"{'ОБЧИСЛЕННЯ РЯДУ ТЕЙЛОРА sin(x) ІЗ ТОЧНІСТЮ eps = 1e-7':^80}")
```

```
    print("=" * 80)
    print(f"| {'x':^8} | {'Сума ряду S(x)':^18} | {'Еталон math.sin':^18} |
{'Ітер.':^6} | {'Похибка':^12} |")
    print("-" * 80)

    test_points = [-3.0, -2.0, -1.0, -0.5, 0.0, 0.5, 1.0, 2.0, 3.0]
    all_within_eps = True
    for x_test in test_points:
        s_val, steps, ref, err = compute_taylor_sin(x_test)
        all_within_eps = all_within_eps and err < EPSILON
        print(f"| {x_test:^8.2f} | {s_val:^18.10f} | {ref:^18.10f} | {steps:^6} |
{err:^12.4e} |")
    print("=" * 80)
    print(f"Похибка в усіх контрольних точках менша за eps = {EPSILON:g}:
{all_within_eps}\n")


if __name__ == "__main__":
    t_start = time.perf_counter()
    main()
    elapsed = time.perf_counter() - t_start
    print(f"Загальний час виконання розрахунків: {elapsed * 1000:.3f} мс\n")
```

5. Результати тестування

Запуск виконувався в середовищі Conda ce_lab_env командами python branching_logic.py та python series_tabulation.py.

5.1. Результат роботи branching_logic.py

```
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab3\lab03> conda activate ce_lab_env
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab3\lab03> & C:\Users\Janis\miniconda3\
envs\ce_lab_env\python.exe "c:/Users/Janis/Desktop/урок/Програмування на Python. Базовий рівень/lab3/lab03/bran
ching_logic.py"
======================================================================
          ВАРІАНТ 2. КУСОЧНО-НЕПЕРЕРВНА ФУНКЦІЯ: РОЗГАЛУЖЕННЯ
======================================================================
|     x      |   Гілка    |       f(x)       |
----------------------------------------------------------------------
|  -3.0000   |  Гілка 1   |    0.08886974    |
|  -1.5000   |  Гілка 1   |    0.37182638    |
|  -1.0001   |  Гілка 1   |    1.54013399    |
|  -1.0000   |  Гілка 2   |   -3.00000000    |
|   0.0000   |  Гілка 2   |    0.00000000    |
|   0.5000   |  Гілка 2   |    1.12500000    |
|   1.0000   |  Гілка 2   |    3.00000000    |
|   1.0001   |  Гілка 3   |    0.04641589    |
|   2.0000   |  Гілка 3   |    1.00000000    |
|   3.0000   |  Гілка 3   |    1.25992105    |
----------------------------------------------------------------------
Розрив у точці x = -1: границя зліва - значення = +4.540302
Розрив у точці x = +1: границя справа - значення = -3.000000
Функція кусочно-неперервна: у точках x = -1 та x = 1 має розриви першого роду.

Коротка схема обчислення (1/x > 1):
  x =  0.0  ->  False
  x =  0.5  ->  True
  x =  2.0  ->  False
======================================================================
Час виконання модуля розгалужень: 1.728 мс
```

Скріншот 1 — Консольний вивід модуля розгалужень

Контрольні точки −1,0001 та 1,0001 потрапляють у першу та третю гілки відповідно, а межові точки −1 і 1 — у другу. Це підтверджує правильність порядку й нестрогості умов. Стрибки функції в точках розриву співпадають з аналітичними оцінками: у точці −1 різниця між лівою границею та значенням становить 1,5403 − (−3) = 4,5403, у точці 1 — 0 − 3 = −3. Вираз 1/x > 1 для x = 0 повертає False без помилки ділення на нуль.

5.2. Результат роботи series_tabulation.py

```
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab3\lab03> conda activate ce_lab_env
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab3\lab03> & C:\Users\Janis\miniconda
3\envs\ce_lab_env\python.exe "c:/Users/Janis/Desktop/урок/Програмування на Python. Базовий рівень/lab3/lab03/
series_tabulation.py"

################################################################################
           ВАРІАНТ 2. ТАБУЛЮВАННЯ ТА РЯД ТЕЙЛОРА ДЛЯ sin(x) (ЛБ №3)
################################################################################

============================================================
      ТАБЛИЦЯ ТАБУЛЮВАННЯ ФУНКЦІЇ f(x) НА [-3, 3], h = 0.25
============================================================
|  №   |     Вузол x      |  Значення y = f(x)   |  Сегмент   |
------------------------------------------------------------
|  1   |     -3.0000      |      0.08886974      |  Гілка 1   |
|  2   |     -2.7500      |      1.28737168      |  Гілка 1   |
|  3   |     -2.5000      |      1.99944942      |  Гілка 1   |
|  4   |     -2.2500      |      1.34300209      |  Гілка 1   |
|  5   |     -2.0000      |      0.34635638      |  Гілка 1   |
|  6   |     -1.7500      |      0.00312619      |  Гілка 1   |
|  7   |     -1.5000      |      0.37182638      |  Гілка 1   |
|  8   |     -1.2500      |      1.00829623      |  Гілка 1   |
|  9   |     -1.0000      |     -3.00000000      |  Гілка 2   |
|  10  |     -0.7500      |     -1.92187500      |  Гілка 2   |
|  11  |     -0.5000      |     -1.12500000      |  Гілка 2   |
|  12  |     -0.2500      |     -0.51562500      |  Гілка 2   |
|  13  |      0.0000      |      0.00000000      |  Гілка 2   |
|  14  |      0.2500      |      0.51562500      |  Гілка 2   |
|  15  |      0.5000      |      1.12500000      |  Гілка 2   |
|  16  |      0.7500      |      1.92187500      |  Гілка 2   |
|  17  |      1.0000      |      3.00000000      |  Гілка 2   |
|  18  |      1.2500      |      0.62996052      |  Гілка 3   |
|  19  |      1.5000      |      0.79370053      |  Гілка 3   |
|  20  |      1.7500      |      0.90856030      |  Гілка 3   |
|  21  |      2.0000      |      1.00000000      |  Гілка 3   |
|  22  |      2.2500      |      1.07721735      |  Гілка 3   |
|  23  |      2.5000      |      1.14471424      |  Гілка 3   |
|  24  |      2.7500      |      1.20507113      |  Гілка 3   |
|  25  |      3.0000      |      1.25992105      |  Гілка 3   |
============================================================
Кількість вузлів: 25  (N = 24 кроків)

================================================================================
            ОБЧИСЛЕННЯ РЯДУ ТЕЙЛОРА sin(x) ІЗ ТОЧНІСТЮ eps = 1e-7
================================================================================
|     x     |    Сума ряду S(x)    |   Еталон math.sin    | Ітер. |    Похибка     |
--------------------------------------------------------------------------------
|   -3.00   |    -0.1411200079     |    -0.1411200081     |   9   |  2.0115e-10    |
|   -2.00   |    -0.9092974265     |    -0.9092974268     |   7   |  3.6423e-10    |
|   -1.00   |    -0.8414709846     |    -0.8414709848     |   5   |  1.5983e-10    |
|   -0.50   |    -0.4794255386     |    -0.4794255386     |   4   |  1.2213e-11    |
|    0.00   |     0.0000000000     |     0.0000000000     |   0   |  0.0000e+00    |
|    0.50   |     0.4794255386     |     0.4794255386     |   4   |  1.2213e-11    |
|    1.00   |     0.8414709846     |     0.8414709848     |   5   |  1.5983e-10    |
|    2.00   |     0.9092974265     |     0.9092974268     |   7   |  3.6423e-10    |
|    3.00   |     0.1411200079     |     0.1411200081     |   9   |  2.0115e-10    |
================================================================================
Похибка в усіх контрольних точках менша за eps = 1e-07: True

Загальний час виконання розрахунків: 2.404 мс

PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab3\lab03>
```

Скріншот 2 — Консольний вивід таблиці табулювання та порівняння ряду з еталоном

Таблиця містить 25 вузлів (N = 24), як і передбачено формулою. Вузли 1–8 належать першій гілці, 9–17 — другій, 18–25 — третій. Перевірка вручну: f(−3) = cos(9) + 1 = 0,08887; f(0,5) = 0,125 + 1 = 1,125; f(2) = $\sqrt[3]{1}$ = 1.

Ряд Тейлора збігається до math.sin(x) за 4–9 ітерацій, найбільша абсолютна похибка складає 3,64 · 10^{-10} (при x = ±2) і в усіх точках менша за ε = 10^{-7}. Для x = 1: 1 − 1/6 + 1/120 − 1/5040 + 1/362880 − 1/39916800 = 0,8414709846, що збігається з результатом програми за 5 ітерацій. Зі зростанням |x| кількість потрібних членів збільшується: 4 при |x| = 0,5 і 9 при |x| = 3.

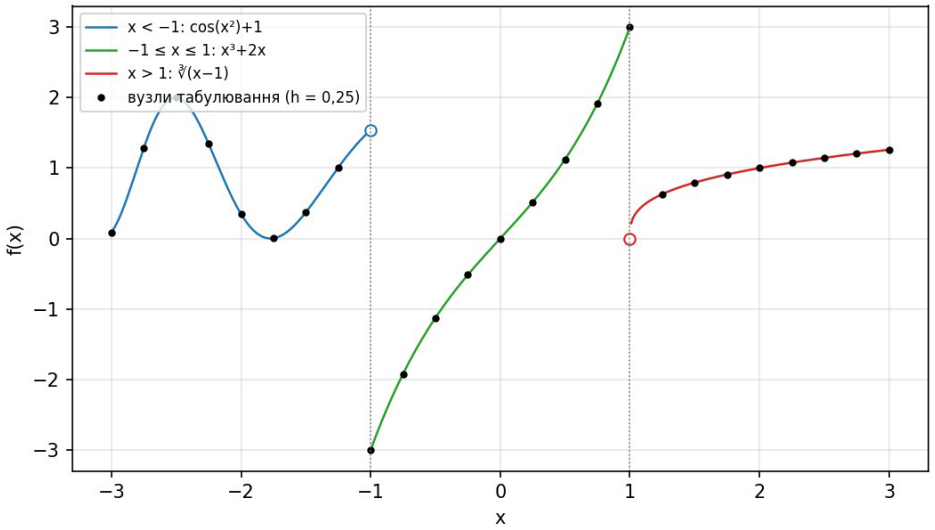


Рисунок 3 — Графік кусочно-неперервної функції f(x) та вузли табулювання

6. Посилання на віддалений репозиторій

Профіль: https://github.com/kefir4ikk

Репозиторій лабораторної роботи :
https://github.com/kefir4ikk/progPython_YevgeniiYatsentiuk/tree/master/labs/lab3

7. Висновки

У ході лабораторної роботи реалізовано програми розгалуженої та циклічної структури для варіанта 2. Каскад if-elif-else забезпечив однозначний вибір однієї з трьох аналітичних гілок функції, а тернарний оператор виявився зручним для компактної класифікації гілки. Коротка схема обчислення

дозволила безпечно перевіряти умови з діленням, не допускаючи ZeroDivisionError.

Табулювання виконано параметричним циклом for з цілочисельним лічильником і формулою $x_i = a + i \cdot h$, що не накопичує похибки округлення: отримано 25 вузлів на відрізку [−3, 3]. Графік і таблиця показують розриви першого роду в точках $x = -1$ та $x = 1$, тож функція є лише кусочно-неперервною.

Ряд Тейлора для sin(x) обчислено циклом while з рекурентним співвідношенням $R(k, x) = -x^2 / (2k(2k + 1))$, виведеним аналітично. Рекурентна формула знижує складність підсумовування з $O(n^2)$ до $O(n)$ і запобігає переповненню факторіалів. Для всіх контрольних точок з діапазону [−3, 3] абсолютна похибка порівняно з math.sin(x) не перевищила $3{,}64 \cdot 10^{-10}$, тобто умову точності $\varepsilon = 10^{-7}$ виконано, а кількість ітерацій не перевищила 9. Обмеження max_iterations захищає програму від нескінченного циклу за межами області збіжності.

Контрольні питання:

1. Поясніть концепцію короткої схеми обчислення (short-circuit evaluation) для логічних операторів and та or. Як вона захищає програму від ділення на нуль при перевірці складних умов?

У виразі A and B, якщо A хибне, операнд B не обчислюється, а результатом є A; у виразі A or B, якщо A істинне, B ігнорується. Тому перевірку небезпечної операції ставлять другим операндом після захисної умови: x != 0 and 1 / x > 1. У лабораторній роботі reciprocal_exceeds_one(0.0) повертає False, не виконуючи ділення.

2. У чому різниця між if-else як інструкцією та тернарним оператором як виразом?

Інструкція if-else виконує дії та не повертає значення; її гілки можуть містити довільну кількість операторів. Тернарний оператор val_true if cond else val_false є виразом: він повертає значення і може входити в інші вирази, генератори, f-рядки та lambda-функції. Він обмежений двома альтернативами (для більшої кількості потрібна вкладеність, що знижує читабельність).

3. Чому при підсумовуванні числових рядів Тейлора на ЕОМ необхідно використовувати рекурентні формули замість прямого обчислення степенів та факторіалів? Який виграш за часовою складністю це забезпечує?

Пряме обчислення k-го члена потребує O(k) множень для x^k та k!, тому сумарна складність становить $O(n^2)$. Рекурентна формула $a_k = a_{k-1} \cdot R(k, x)$ виконує сталу кількість операцій на крок, і сумарна складність дорівнює O(n). Крім того, велика проміжна величина x^{2k+1} та (2k + 1)! можуть переповнити тип float і призвести до втрати точності, тоді як члени ряду, обчислені рекурентно, залишаються помірними.

4. Опишіть специфіку роботи блоку else у циклічних конструкціях while та for мови Python. За яких умов блок else буде проігноровано інтерпретатором?

Блок else виконується, якщо цикл завершився природно: умова while стала хибною або ітератор for вичерпався. Якщо цикл перервано оператором break (а також return чи винятком), блок else пропускається. У compute_taylor_sin оператора break немає, тому else виконується завжди, і всередині нього перевіряється, чи зупинку спричинено лімітом ітерацій.

5. Чому функція range() не підтримує дійсні числа (float) як крок генерації step? Яким чином реалізується коректна координатна сітка для табулювання дійсних функцій?

range() моделює арифметичну прогресію цілих чисел з точною довжиною та сталою пам’яттю. Для дійсного кроку довжина послідовності залежала б від похибок округлення (наприклад, чи потрапить межа b у послідовність). Тому перебирають цілий лічильник i = 0..N і обчислюють $x_i = a + i \cdot h$, де $N = \lfloor (b - a) / h \rfloor$, як зроблено в tabulate_function.

6. Що таке інваріант циклу та умова збіжності ітераційного процесу? Як запобігти виникненню нескінченного циклу при обчисленні числових рядів за допомогою while?

Інваріант циклу — твердження, істинне перед кожною перевіркою умови та після кожної ітерації. Тут: series_sum = $\Sigma_{j=0}^{k} a_j$, $a_k = (-1)^k x^{2k+1} / (2k + 1)!$. Необхідна умова збіжності ряду — $a_k \to 0$; ряд для sin(x) збігається для всіх x, бо $|R(k, x)| \to 0$ при $k \to \infty$. Щоб уникнути зациклення, потрібно, щоб змінні умови змінювалися на кожному кроці (k зростає, $|a_k|$ спадає), додати ліміт max_iterations, перевіряти $\varepsilon > 0$ та працювати лише в області збіжності ряду.